Data Visualization and Exploration

COMS7056A/COMS4060A

Muhammad Nasir

Lecture 6

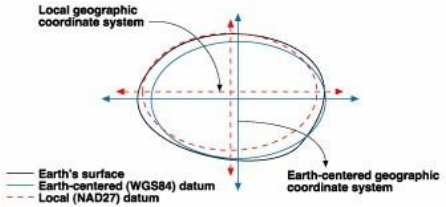
Lesson Plan

- Geospatial data
  - Basics
  - Plotting
  - Exploration

Geospatial data

- Geospatial data describes information about a specific location on the Earth's surface
  - GPS coordinates
  - Addresses
  - Boundaries of areas - borders of countries, provinces, etc
- However, it isn't *only* location data – there can be many additional features or attributes that further describe that location
- Geospatial data is usually processed and analysed using a geographic information system (GIS) – systems and programs designed specifically to deal with geospatial data
  - Geopandas in python
  - PostGIS – extension to postgres that adds additional functionality
- Typically, when working with geospatial data, we will work with the following:
  - **Vector data**: graphical representations of the world, using mostly collections of points, lines, and polygons. This would be things like road networks or boundaries of locations
  - **Raster data**: images (or grids of pixels), where each pixel has a value representing information about the location – a colour or unit of measurement. Satellite imagery on a map is an example of raster data
  - **Attributes**: Any additional information (non-spatial data), that describing a feature at a location. Eg. A map displaying buildings, where each of the buildings, in addition to their location, has additional attributes such as the type of use (housing, business, government, etc.), the year it was built, and how many stories it has

Geographic coordinate systems



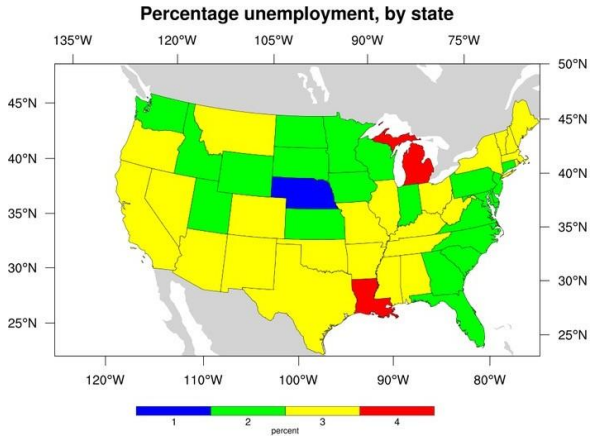
- This is the 3D spherical surface used to define locations on the earth
- However, there are multiple models or approximations of the spheroid that described the Earth, each having their own definition of:
  - Datum: position of the spheroid relative to the centre of the Earth
  - Prime meridian: where the zero degree line of longitude is (typically Greenwich)
  - Angular unit of measure: usually degrees or radians
- Coordinates in one datum or GCS do not necessarily refer to the same point on Earth, as they are in reference to the origin of the datum
  - Typically WGS84 is used as a global standard
  - Local datums like NAD1927 provide better approximations in specific local areas (like North America)
- EPSG Geodetic Parameter Dataset is a registry of datums and GCSs, so each datum has a unique EPSG code:
  - E.g. WGS84 is the same as EPSG:4326
- Make sure you are using the correct geographic coordinate system when plotting your data!
  - Best to convert everything to EPSG:4326

Coordinate formats

- Format of coordinates
  - Decimal degrees: degrees and fraction of degrees: DDD.DDDDD
  - Sexagesimal degree: degrees, minutes and seconds: DDD MM.MMM’ SS.S”
  - Degrees and decimal minutes: degrees, minutes, and fractions of minutes: DDD MM.MMM’
- Longitude:
  - North will be indicated by N or “+”
  - South will be indicated by S or “-”
- Latitude:
  - East is E or “+”
  - West is W or “-”
- Example: Wits coordinates
  - 26°11'34.0"S 28°01'50.0"E
  - -26.192779, 28.030561

Shapefiles

- One of the most popular formats for geospatial data, which is synonymous with vector data
- It describes features using Points, Lines and Polygons
- Shapefile is actually a collection of files, with at least:
  - .shp: geometry of features
  - .shx: index for the feature geometry
  - .dbf: attributes for each shape
- Municipal borders, city boundaries, etc are typically available as shapefiles for download
  - This is useful to add as a layer to any plotting you may want to do
  - https://egis.environment.gov.za/ for example
- If you need/want to create your own, you can do this from Google Earth for example, then export your shapefile



Distances

- Believe it or not, the Earth is not flat… but it's not perfectly spherical either
- If you look at two points close to each other, you can approximate it as being flat, but as the distance increases, the curvature adds distance – great circle distance.
- You can use the haversine formula or Lambert's formula to get a more accurate value.
- Keep in mind that distances represented by a single degree in long/lat change based on the location!

φ	Δ^1_{lat}	Δ^1_{long}
0°	110.574 km	111.320 km
15°	110.649 km	107.551 km
30°	110.852 km	96.486 km
45°	111.133 km	78.847 km
60°	111.412 km	55.800 km
75°	111.618 km	28.902 km
90°	111.694 km	0.000 km

Plotting

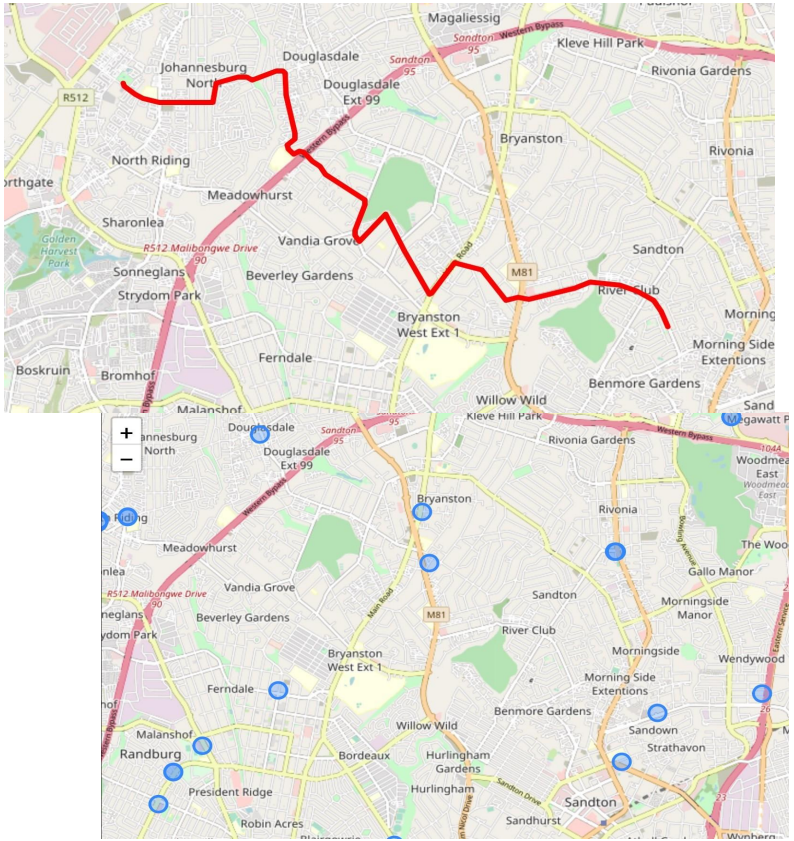
- Plotting coordinates by themselves is probably not useful – you want to plot them together with some additional information… ie. Where does it exist in the world?
  - Is it on land? In the sea?
  - Which country is it in? Which province? \
  - Plot it on a map, basically… (Check out folium in python, or visit the mapbox website to check this out)
- We achieve this by plotting a number of layers on a map, consisting of a mixture of vector data and raster data, and overlaying or summarise the attributes of the data
- Depending on the application, this can be a map of:
  - satellite imagery (Google Earth, EOS, etc.),
  - country/provincial borders
  - road networks
- We get greater context of our data, and what it *means*
- Most importantly - this will pick up obvious issues in the data too!
  - Where we get nulls or zeroes in non-geospatial data, we often find coordinates of (0, 0)
  - The other thing this helps with is to check if the coordinates are in the correct hemisphere

LAPD Crime stats example

- The LAPD releases their crime stats online showing which areas have the highest rate of crime
- Between Oct 2008 and March 2009, 1380 (4%) of all crimes that were uploaded were all within a single block in a good neighbourhood… Right where the LAPD HQ was!
- When crimes are reported in LA, a location is required, but, if the location could not be parsed by the mapping software, it would automatically set the location as the LAPD HQ, severely skewing results
- Their fix was to rather store the location as (0, 0)… Which has its own issues

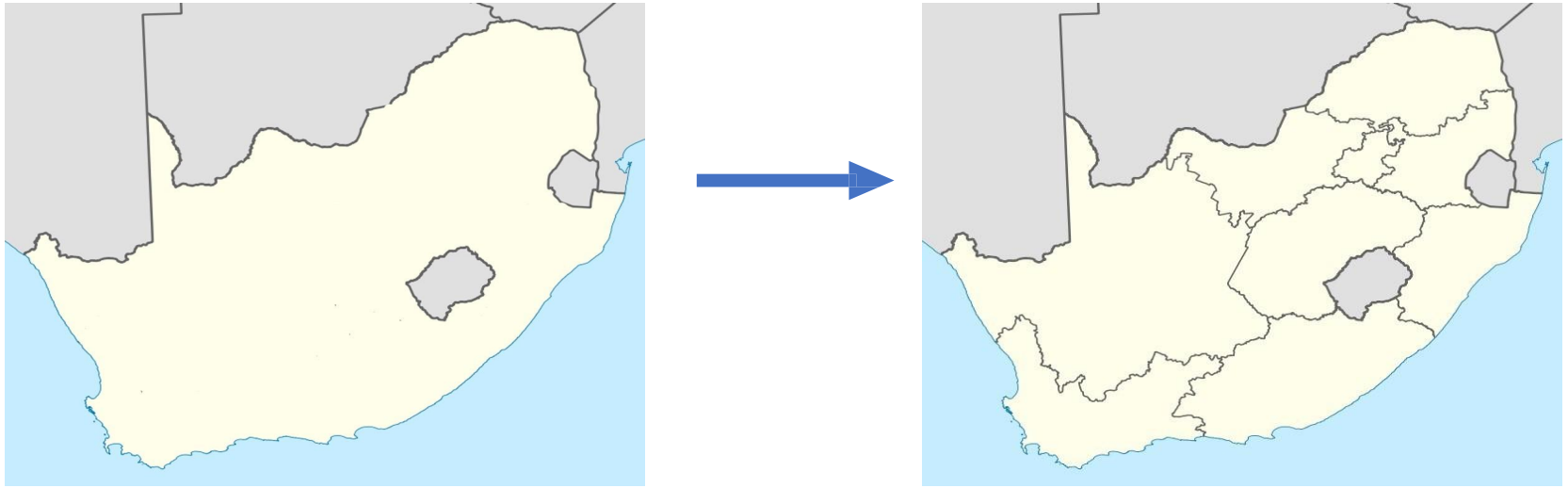
Plotting on a map

- Plotting the points on a map immediately gives context in this situation
  - The map in this case is vector data, showing roads and suburbs in a lot of detail
- In the top image, the data points are *sequential* and represent a trajectory
- In the bottom image, we have a number of seemingly randomly scattered points
  - Importantly, there are not many data points, so it is easy for us to plot them individually



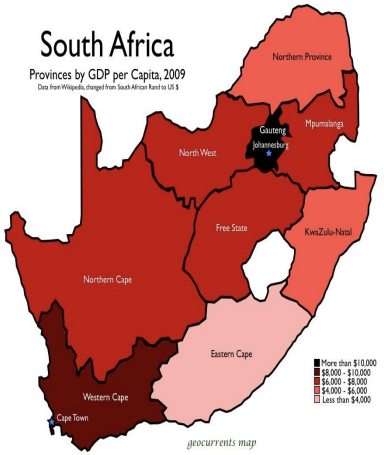
Shapefiles

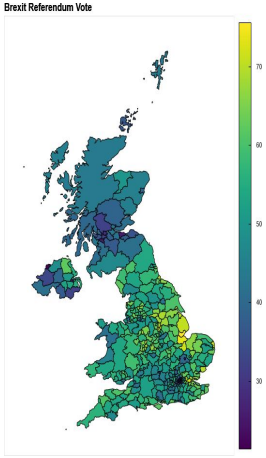
However, sometimes you do not want or need as much granularity as in the previous example, say you want to look at the provinces of SA – you don't want every single road shown here



Plotting - Choropleth

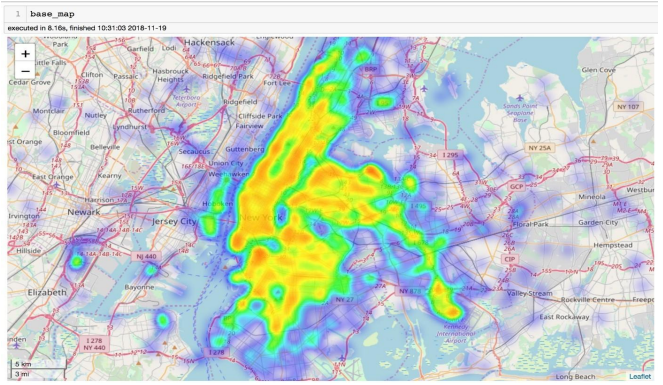
- If you have many data points it might not be useful or feasible to plot all of them at once (like a scatter plot), so you can plot a choropleth instead
- It is essentially like plotting a histogram onto a map
- You have the map with some way of segregating it into smaller areas (like provinces), and you want to know how some statistic or metric is distributed between these areas (like GDP, for example)
- Areas of a map are shaded according to an aggregation of the statistic



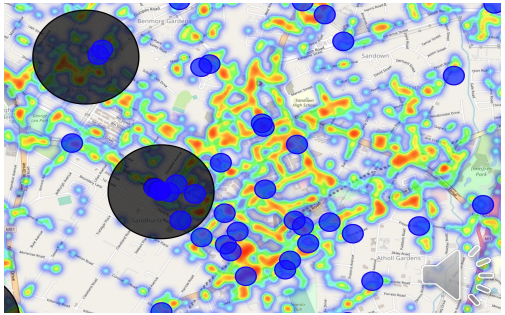


Plotting – heatmap

- Often it is useful to plot some aggregation displaying intensity or some aggregation of a metric, but you want it on a broader scale than a choropleth allows – use a heatmap
- The colour represents the intensity, and will change as you zoom in and out



2 months of cab rides in NYC

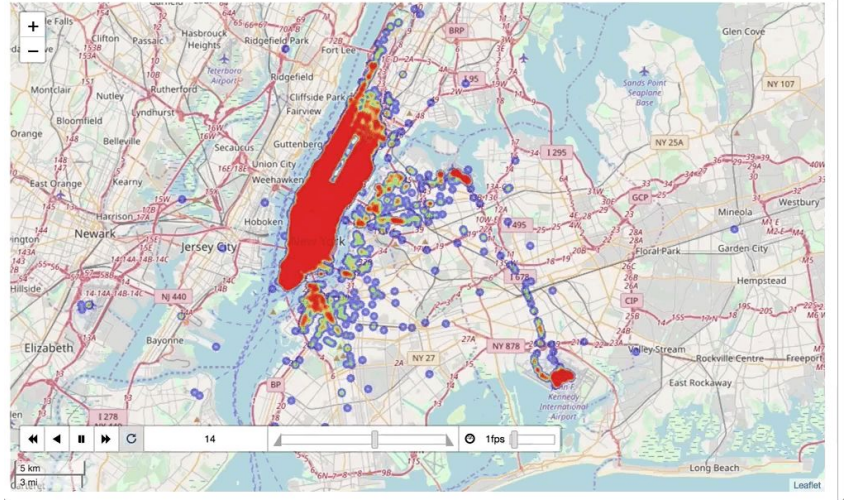


Traffic in Sandton, with additional markers

Adding time

- You might need/want to add a time component to the map – how does the distribution change throughout the day?
  - As an example, how does the location of taxi locations change in NYC by hour of day?
- Here, you can consider:
  - restricting the dataset to specific time periods, and creating a plot for each time period
  - Creating an animation that runs through each time period

```
base_map = generateBaseMap(default_zoom_start=11)
HeatMapWithTime(df_hour_list, radius=5, gradient={0.2: 'blue', 0.4: 'lime', 0.6: 'orange', 1: 'red'}, min_opacity=
base_map
```



https://towardsdatascience.com/data-101s-spatial-visualizations-and-analysis-in-python-with-folium-39730da2adf

Plotting - multiple methods

- You can add multiple methods together to create more informative maps!
- Consider the one on the top,
  - The location of taxis is plotted as a scatter plot, with the timestamp determining the colour of the marker
  - The data shown underneath is a histogram of when all the trips took place, by hour of day
- This is an interactive one actually, check it out here https://dash-gallery.plotly.host/dash-uber-rides-demo/

Other attributes

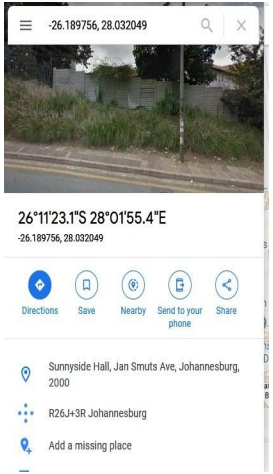
- So far, we have looked at attributes like density and GDP, but the attributes can be any sort of numerical feature
- Here is an example showing the road network of San Francisco with a heatmap showing the incline of the roads
  - This allows for an easy way to identify where the steepest streets are in downtown SF
- Check out the source code here https://github.com/gboeing/osmnx-examples/blob/v0.11/notebooks/12-node-elevations-edge-grades.ipynb

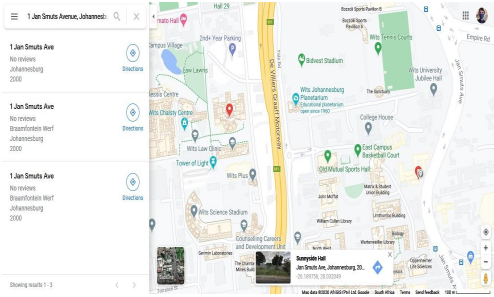
Geofencing

- Geofencing is a way of creating a virtual boundary around a location
  - Shapefiles essentially contain a bunch of these boundaries, but they are used to determine municipal boundaries, country borders etc.
- GPS coordinates are then checked to determine if they are within this boundary
- This has many applications
  - Apple/Google location-based reminders
  - Vehicle security system
  - Family tracking (Life365 for example)
- Given some coordinate, it is often useful (or necessary) to know in which shape (polygon in the shapefile) the point lies
  - In python, the shapely package has a method called .isin() that lets you determine if this is true
- Similarly, you might need to know which points lie within a given range around a point
  - E.g. Strava has safety feature that does not display any data within X metres around your house – a “safety zone”

Geocoding

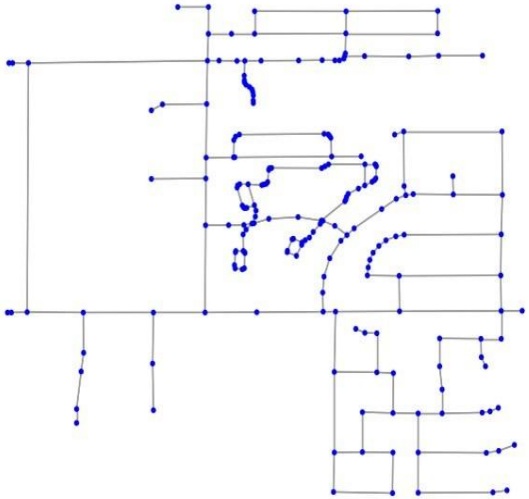
- Geocoding takes an address (in text) and returns the coordinates of the location
- Reverse geocoding does the opposite, by taking coordinates and finding a name or address for the location
- This useful if you want to do some form of text-based analysis on the addresses, or, conversely, you have addresses and would like to plot them on a map
- You can do this with openstreetmaps for free and write your own code to do it, or use:
    - Mapbox gives you a few thousand free per month
    - Google also provides a sort of free service
- This can be tricky to work with
    - Similar addresses might give an incorrect location: e.g. there is an Outspan Road in River Club and in the South of Johannesburg
    - Some scepticism here is always a good idea

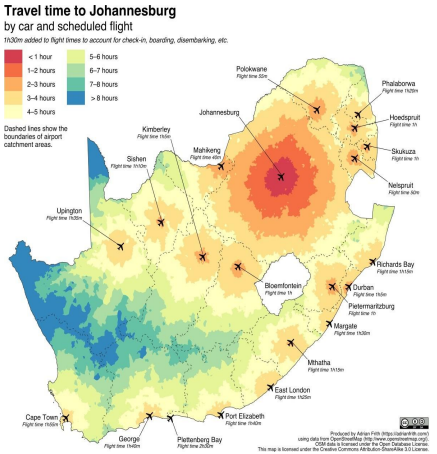




Network Objects

- Geometric network objects are used to model interconnected features, like road and transportation networks, for example
  - Has a number of nodes, connected in some way, and often with weights and direction
- Visualising and analysing the road networks is useful in many circumstances
  - Routing, planning
- Road networks can be overlaid on a map as an additional layer, purely for display, or they can be used for further analysis and exploration
  - How close are you to a highway, a gravel road, etc?
  - How long would it take someone to go from A to B? Does this explain some clustering in one area?
  - How far away can someone travel from point A in any direction in a given time?
  - What is the speed limit at some location?





Spatial Autocorrelation

- Conventional exploratory data analysis does not investigate locations explicitly, focusing on relationships and correlations between features in the dataset
- Exploratory Spatial Data Analysis (ESDA) correlates features in the data to a location(s), taking into account the values of the feature in the surrounding area
- We use spatial autocorrelation to do this investigation and analysis:
  - Spatial autocorrelation describes the presence (or absence) of spatial variations in a given variable
  - It gives us a way to visualise the associations between some spatially ordered values
  - E.g. prices and spatially lagged houses

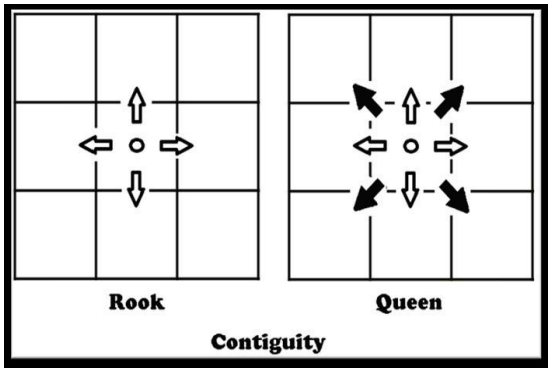
https://mgimond.github.io/Spatial/spatial-autocorrelation.html

Spatial Autocorrelation

- There are two main aspects to spatial autocorrelation: spatial similarity and attribute similarity
- Initially, the entire location area must be divided up into spatial units
  - These can be square blocks of equal area, for example
- **Spatial similarity:** This is formalised by using a matrix of spatial weights, where weights between neighbouring locations are determined based on either a distance metric or a contiguity-based approach
  - Contiguity-based weight matrices are defined, in their simplest form the weights are set to 1 or 0
  - Common contiguity matrices are the Rook and Queen

Weight matrix of size *nxn* with an entry for each observation – w_{ij} is non-zero if two points are neighbours

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n1} & w_{n2} & \dots & w_{nn} \end{bmatrix}$$

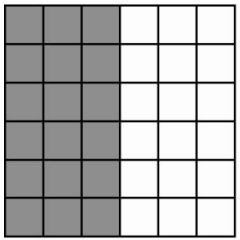


Spatial Autocorrelation

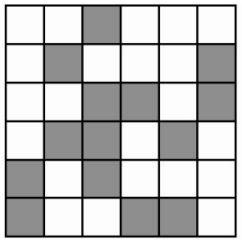
- **Attribute similarity:** spatial weights allow for spatially explicit variables that are functions of attributes for each location
  - Spatial Lag (summarised neighbourhood value) is the product of the spatial weights matrix for a given variable (eg. Price, population or income), it essentially standardizes the rows and takes the average result in each weighted neighborhood
    - $y\_lag_i = \Sigma w_i y_i$
  - Spatial rate smoothing is similar to the lag, but is used to determine how a rate is influenced by its neighbours:
    - $y = \Sigma w_{ij} O_j / \Sigma w_{ij} P_j$, where O is the count (e.g. number of people affected in the area) and P is the total population in the area
- This has the effect of smoothing out the value similarity locally
- How to associate the value of the attribute with the spatial lag values?

Spatial Autocorrelation

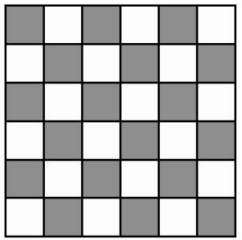
- Ideally, we are looking for an n*n grid showing us the correlation between neighbours
  - Highly positive: similar to its neighbours
  - High negative: completely different from neighbours
- We can look at this globally and locally
  - The global spatial autocorrelation focuses on the overall trend in the dataset and tells us if the degree of clustering in the dataset.
  - The local spatial autocorrelation detects variability and divergence in the dataset, which helps us identify hot spots and cold spots in the data.



Positive spatial autocorrelation



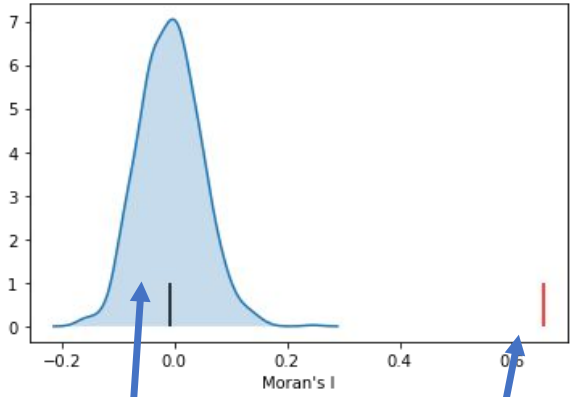
No spatial autocorrelation



Negative spatial autocorrelation

Global Spatial Autocorrelation

- This determines the overall pattern in the dataset
- We are investigating if there is a trend, which is summarised in much the same was as correlation coefficient
    - Moran's I statistics is typically used to determine the global spatial autocorrelation
- Critically, we want to know if the spatial distribution is different from what we would expect if the process generating the spatial distribution were completely random one
- When testing to see if the correlation is significant, the null hypothesis of complete spatial randomness (CSR) is used.
- A number of random distributions are created (like hundreds or thousands) to construct a reference distribution to evaluate the statistical significance of observed data



Complete spatial randomness distribution

Statistically significant Moran's I, showing correlation

Moran’s I statistics

- Moran's I is a test for global autocorrelation for a continuous attribute:

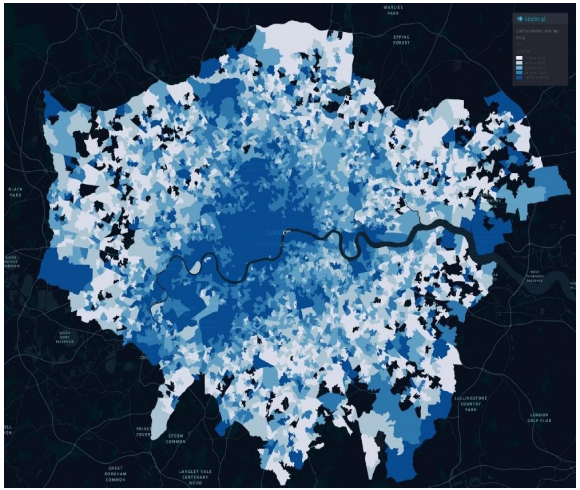
$$I = \frac{N}{W} \frac{\sum_{i=1}^{N} \sum_{j=1}^{N} w_{ij}(x_i - \bar{x})(x_j - \bar{x})}{\sum_{i=1}^{N} (x_i - \bar{x})^2}$$

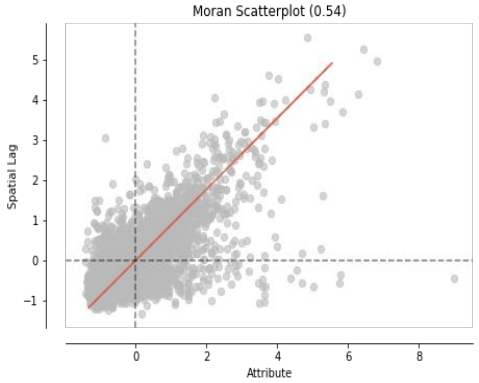
- Where:
  - I is between -1 and 1
  - -1 = negative spatial correlation
  - 0 = no spatial correlation
  - 1 = positive spatial correlation

Airbnb in London

- Here we plot the prices of Airbnb rentals throughout London as a choropleth
- It *looks* like there are clusters in the data, or at least some kind of pattern… we can quantify this with spatial autocorrelation
- Moran scatterplot shows the distribution of the spatial lag of a variable and its actual value… Here, we have an I of 0.54, indicating there is a positive correlation

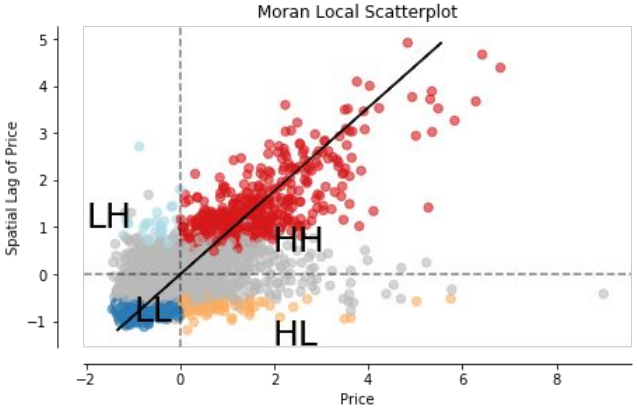
https://towardsdatascience.com/what-is-exploratory-spatial-data-analysis-esda-335da79026ee





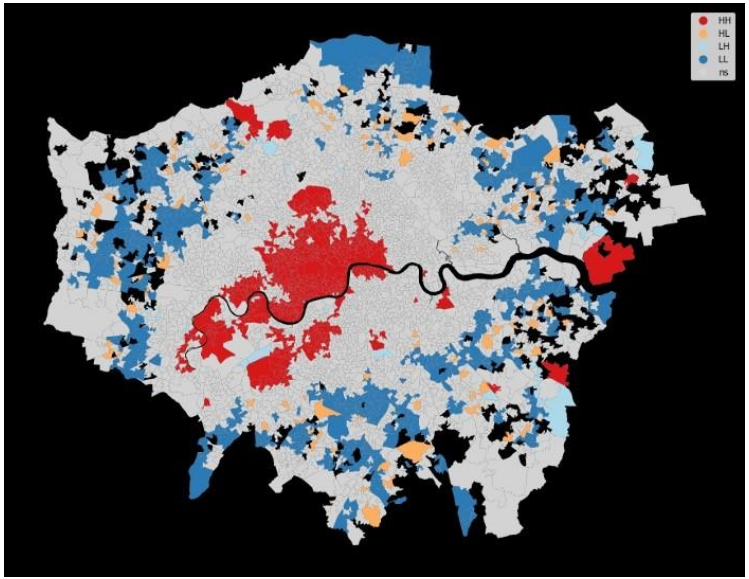
Moran’s I local

- We want to know where the clusters are in the data – we know there is a trend in the data, but where are there local patterns?
- The local spatial autocorrelation detects variability and divergence in the dataset, which helps us identify hot spots and cold spots in the data
- Local Indicators of Spatial Association (LISA) is used to identify the local clusters. LISA classifies areas into four groups: high values near to high values (HH), Low values with nearby low values (LL), Low values with high values in its neighbourhood, and vice-versa. I is calculated for each spatial unit independently



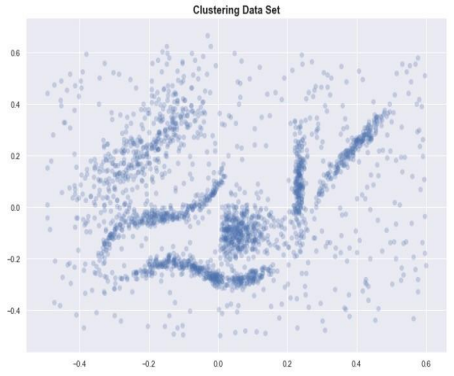
Local spatial autocorrelation

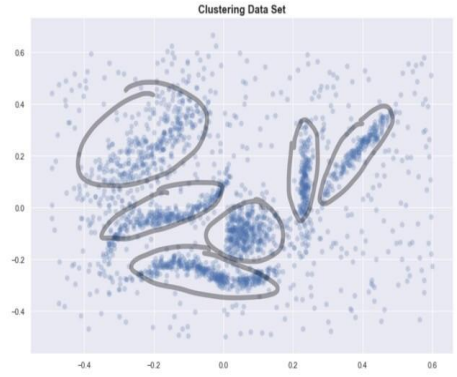
- The different areas categorised by LISA
- This has given us insights into where to look for clusters and investigate further!



HDBSCAN

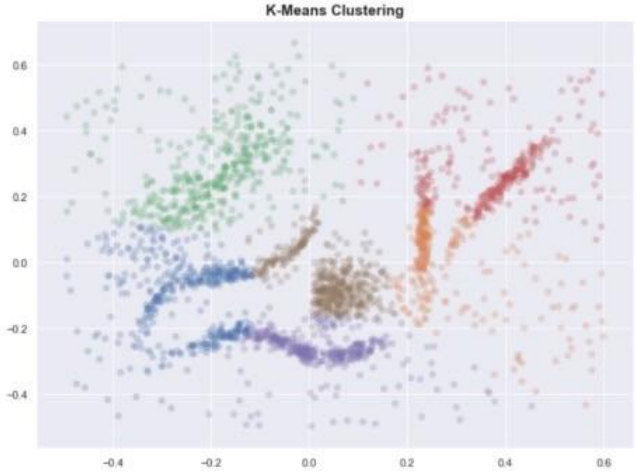
- Another way of identifying clusters is to cluster directly with algorithms like k-means
  - However, these often require you to guess how many clusters there are beforehand…
  - We want a way to do this unsupervised!
- Clustering like K-means often produces rigid shapes, but using
- density-based methods often work well even when the data isn't clean and the clusters are weirdly shaped
  - DBSCAN is a popular density-based spatial clustering algorithm
- The upgraded one is HDBSCAN - Hierarchical Density-Based Spatial Clustering of Applications with Noise is one tool we can use
  - Essentially this performs DBSCAN with many different parameters and integrates the result to find a clustering that gives the best stability over all parameters
  - This allows HDBSCAN to find clusters of varying densities and be more robust to parameter selection.
- How does HDBSCAN do this? At a high level, we can simplify the process of density-based clustering into these steps:
  1. Estimate the densities
  2. Pick regions of high density
  3. Combine points in these selected regions

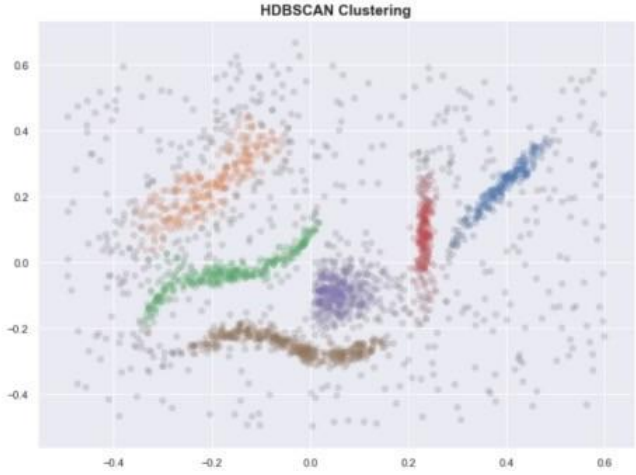




https://nbviewer.jupyter.org/github/scikit-learn-contrib/hdbscan/blob/master/notebooks/How%20HDBSC
https://towardsdatascience.com/a-gentle-introduction-to-hdbscan-and-density-based-clustering-5fd79329c1e8
https://jakevdp.github.io/PythonDataScienceHandbook/05.11-_k-means.html

K-means vs HDBSCAN





Resources and tools

- PostGIS
- Tools for python:
  - Geopandas
  - Osmnx
  - Folium
  - Pysal
  - Shapely
  - Fiona
- Mapbox
- Plotly